

Week 5: Infrastructure as Code & Deployment

Infrastructure as Code, CI/CD & Deployment Engineering Internship Guide

Week 5: Infrastructure as Code & Deployment

The Complete Theoretical Intern Training & Implementation Guide

Welcome to the Week 5 Infrastructure as Code & Deployment Intern Guide. This week marks the consolidation of your cloud engineering capabilities. You will learn how to automate the provisioning, testing, deployment, and monitoring of serverless architectures using Infrastructure as Code (IaC) and Continuous Integration / Continuous Delivery (CI/CD) pipelines on Amazon Web Services (AWS).

This guide serves as a self-contained manual. It contains no code snippets (except for step-by-step shell execution commands, file trees, and architecture diagrams), focusing instead on explaining every IaC parser, pipeline stage, GitHub workflow event, monitoring alert, and trace mechanism from first principles. By reading this guide, you will understand exactly what was built during Week 5, how the components interact, and how to deploy and test the architectures step-by-step.

Table of Contents

1. Module 1: Infrastructure as Code (IaC) Foundations
2. Module 2: AWS Serverless Application Model (SAM)
3. Module 3: Continuous Integration & Continuous Delivery (CI/CD)
4. Module 4: Environment & Configuration Management
5. Module 5: Serverless Monitoring & CloudWatch Alarms
6. Module 6: Distributed Tracing with AWS X-Ray
7. Module 7: Serverless Logging Best Practices
8. Module 8: The SAM-Managed Serverless Infrastructure Architecture
9. Module 9: Resource Definitions: SAM YAML vs. Terraform HCL
10. Module 10: Hands-On Lab & Project Setup Guide
11. Module 11: Week 5 Learning Conclusion

Module 1: Infrastructure as Code (IaC) Foundations

Managing cloud resources through the AWS Web Console is prone to human error, difficult to audit, and impossible to reproduce reliably across environments. Infrastructure as Code (IaC) solves this by defining infrastructure declaratively in text files.

1.1 Benefits of IaC

- **Consistency:** Ensures the exact same resources are deployed in every stage (development, staging, production).
- **Speed:** Automates resource creation, reducing deployment time from hours to seconds.
- **Auditability:** Infrastructure files are checked into Git, providing a complete history of who changed what and when.
- **Safety:** Allows you to run dry-run validation checks before making changes, preventing accidental deletions.

1.2 AWS CloudFormation Basics

AWS CloudFormation is the native engine that reads declarative JSON or YAML templates and builds the specified resources in order, handling resource dependencies automatically.

- **Template:** A text file defining the resources you want to create (such as S3 buckets or Lambdas).
- **Stack:** A single logical unit of AWS resources managed together. If you delete a stack, CloudFormation deletes all resources created inside it.
- **Rollbacks:** If CloudFormation encounters an error while deploying a stack (e.g., a service limit error), it aborts the deploy and rolls back, reverting all resources to their previous state to prevent broken deployments.

Module 2: AWS Serverless Application Model (SAM)

While CloudFormation is powerful, its syntax is verbose. Defining a simple Lambda function with an API Gateway path can require hundreds of lines of YAML. To simplify this, AWS developed the AWS Serverless Application Model (AWS SAM).

2.1 The SAM Transform Header

SAM is a macro extension of CloudFormation. A SAM template starts with a header:

```
`Transform: AWS::Serverless-2016-10-31`
```

When CloudFormation processes this template, it expands the simplified SAM resources into full CloudFormation configurations.

2.2 Core SAM Resources

- **AWS::Serverless::Function:** Defines a Lambda function, its runtime, environment variables, execution roles, and event triggers.
- **AWS::Serverless::Api:** Explicitly defines a REST API Gateway, configuration stages, and endpoint routes. If you do not define this resource, SAM creates one implicitly when you attach API triggers to a function.
- **AWS::Serverless::SimpleTable:** Provisions a DynamoDB table using a single Partition Key (`id` by default), utilizing On-Demand capacity to simplify configuration.

Module 3: Continuous Integration & Continuous Delivery (CI/CD)

DevOps is the integration of software development (Dev) and IT operations (Ops) to build, test, and release code faster. CI/CD pipelines automate this process:

- **Continuous Integration (CI):** Automates building the code, checking for syntax errors (linting), and running unit tests whenever a developer pushes a change to Git.
- **Continuous Delivery (CD):** Automatically deploys the tested code to staging or production environments.

3.1 AWS Native Pipelines vs. GitHub Actions

- AWS CodePipeline: Orchestrates the pipeline stages (Source -> Build -> Deploy).
- AWS CodeBuild: A managed build service. It spins up a temporary Docker container, runs compilation and test scripts defined in a `buildspec.yml` file, packages the artifacts, and shuts down.
- GitHub Actions: A developer-centric CI/CD framework integrated directly into GitHub. Workflows are defined in YAML files under `.github/workflows/`. It uses runners to execute tasks in response to events (e.g., git push or pull requests).

Module 4: Environment & Configuration Management

Enterprise applications maintain separate, isolated environments:

- Development (Dev): Used by developers to deploy and test code iterations.
- Staging (Testing): A copy of the production environment used for integration testing.
- Production (Prod): The live environment accessed by end users.

To manage configurations between environments, you use SAM Config Files (`samconfig.toml`). This file stores default deployment arguments (such as stack names, S3 deployment buckets, and regions) for each environment stage. During deployment, you specify the target stage to override configuration parameters without modifying the code templates.

Module 5: Serverless Monitoring & CloudWatch Alarms

Monitoring in serverless architectures must be automated because compute containers are short-lived.

5.1 CloudWatch Metrics and Logs

Lambda publishes metrics to CloudWatch automatically, including:

- Invocations: Total execution runs.
- Errors: Total failed executions.
- Duration: Execution runtime in milliseconds.
- Throttles: Executions blocked because concurrent run limits were exceeded.

5.2 CloudWatch Alarms Configuration

An alarm monitors a metric over a specified duration and triggers notifications if the metric exceeds a threshold.

- Statistic: The calculation applied to the metric (e.g., `Sum` of Errors).
- Period: The time window (in seconds) used to calculate the statistic (e.g., check every 60 seconds).
- Evaluation Periods: The number of consecutive periods the metric must exceed the threshold before the alarm transitions to the `ALARM` state.
- Treat Missing Data: Determines what to do if no data is logged during a period (e.g., treat missing data as `notBreaching` to prevent false alarms).

Module 6: Distributed Tracing with AWS X-Ray

In a microservices architecture, a single user request can trigger a chain of services (e.g., API Gateway -> Authorizer Lambda -> SQS Queue -> Processor Lambda -> DynamoDB -> S3). If the request is slow, finding which service caused the delay is difficult.

AWS X-Ray resolves this by providing end-to-end tracing:

- It generates a unique trace ID and passes it in the request headers through every service boundary.
- Each service records its start time, execution duration, and status code, reporting them back to X-Ray.
- Service Map: A visual representation showing the dependency graph and latency bottlenecks of your architecture.

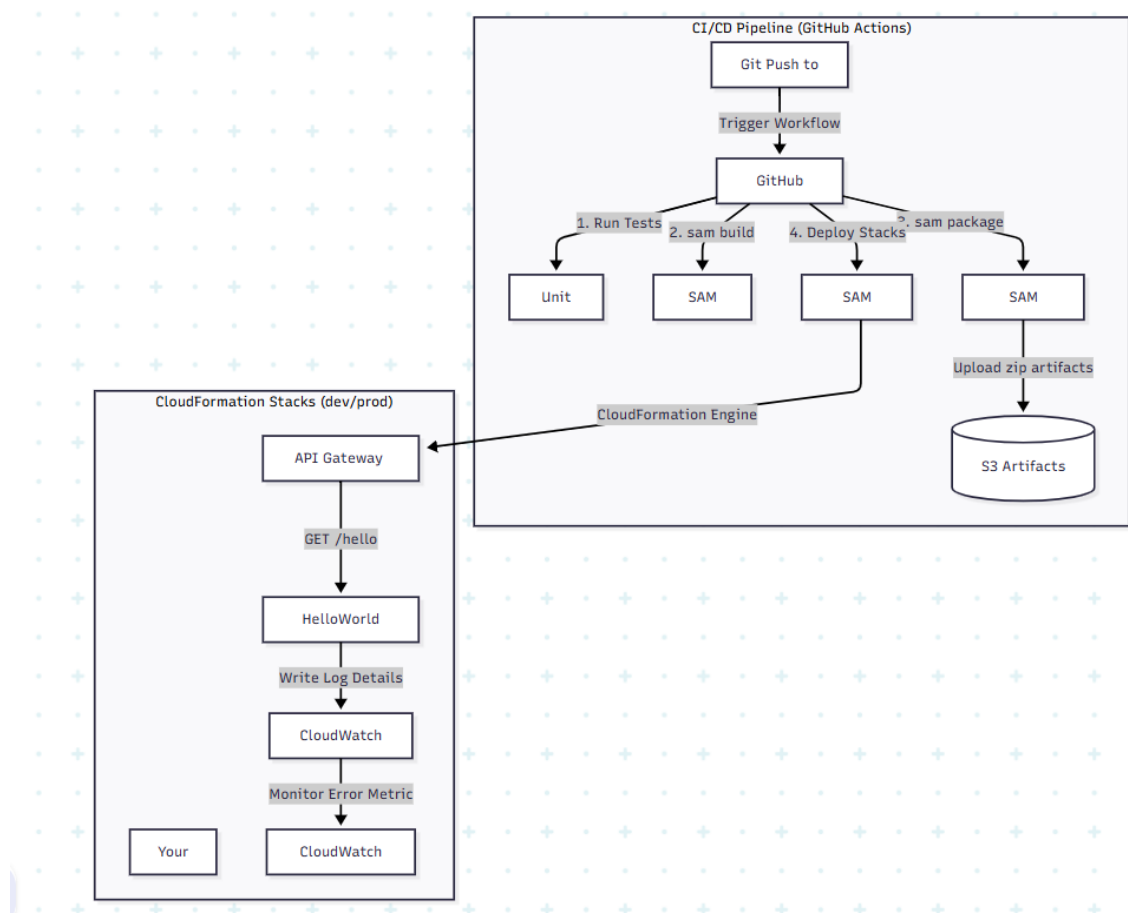
Module 7: Serverless Logging Best Practices

Logging is critical for troubleshooting serverless systems.

- **Avoid Print Statements:** Standard print statements output unstructured text, which is difficult to search.
- **Structured Logging:** Log outputs as JSON dictionaries containing structured fields (e.g., `timestamp`, `log\_level`, `request\_id`, `message`). This allows CloudWatch Logs Insights to parse, filter, and run SQL-like queries on your logs.
- **Log Retention:** By default, CloudWatch retains logs forever. To manage costs, you should set a log retention policy (e.g., automatically expire logs after 14 days).

Module 8: The SAM-Managed Serverless Infrastructure Architecture

This week's assignment deploys a secure, monitored API Gateway endpoint using AWS SAM, automated by a multi-stage GitHub Actions CI/CD pipeline.



- **Development Workflow:** The developer pushes changes to a feature branch. GitHub Actions builds the code and deploys it to a dynamic feature stack for testing.
- **Production Workflow:** Pushing to the `main` branch triggers the full pipeline. The runner runs unit tests, packages the code, uploads the artifacts to S3, and deploys the stack to the `dev` stage. After verification, it deploys the changes to the `prod` stage.
- **Monitoring Flow:** If the Lambda function encounters an exception during execution, CloudWatch logs the error metric, triggering the alarm to alert operations.

Module 9: Resource Definitions: SAM YAML vs. Terraform HCL

Comparing resource definitions highlights the structural differences between these tools:

- **AWS Lambda Function:**
- **Terraform (HCL):** Requires declaring separate resources for `aws\_lambda\_function`, `aws\_iam\_role`, `aws\_iam\_policy\_attachment`, and `data.archive\_file` to zip the code.
- **SAM (YAML):** Declares a single `AWS::Serverless::Function` resource. SAM automatically packages the zip file, creates the IAM execution role, and configures the API Gateway trigger path under its properties.
- **API Gateway:**
- **Terraform (HCL):** Requires declaring resources for REST APIs, HTTP methods, paths, integrations, deployments, and permissions separately.
- **SAM (YAML):** Set up automatically when you configure an `Api` trigger under the function's event properties.

Module 10: Hands-On Lab & Project Setup Guide

This module details the directories, execution files, and configurations required to deploy and run the Week 5 SAM application and CI/CD pipelines.

10.1 Project File Directory Structure

Below is the directory tree containing the code resources for this week's assignments:

```
c:\Users\Vaibhav.S.P\Downloads\TringApps Intern
├── .github/
│   └── workflows/
│       └── pipeline.yaml          # GitHub Actions multi-stage CI/CD configuration
└── Week 5/
    ├── template.yaml             # AWS SAM configuration template
    ├── samconfig.toml            # Stage overrides configuration
    ├── hello.py                  # Hello World Python Lambda handler
    └── Image Uploads II/         # Advanced decoupled pipeline reference directory
```

10.2 Step-by-Step Implementation Runbook

Follow these steps to deploy and test the components of Week 5:

Task A: AWS CLI & SAM CLI Setup

- Verify the SAM CLI is installed on your local machine:

```
sam version
```

- Verify your terminal is authenticated with your AWS account:

```
aws sts get-caller-identity
```

- Navigate to the project directory:

```
cd "Week 5"
```

Task B: Local Build and Verification

- Run the build command to package dependencies and prepare local artifacts:

sam build

Expected Output: Creates a ``.aws-sam`` directory containing the built code and CloudFormation templates.

- Test the function locally:

sam local invoke HelloWorldFunction

Expected Output: Docker spins up a local container and runs the handler, returning ``statusCode: 200`` and the response body.

Task C: Manual Deployment

Deploy the stack manually to configure your environment settings:

- Run the guided deployment wizard:

sam deploy guided

- Respond to the prompts:
 1. Stack Name: ``sam-hello-world-dev``
 2. AWS Region: ``us-east-1``
 3. Confirm changes before deploy: ``y``
 4. Allow SAM CLI IAM role creation: ``y``
 5. Save arguments to configuration file: ``y`` (This creates or updates ``samconfig.toml``).
- Verification: Capture the output API endpoint URL and test the endpoint:

curl https://xxxx.execute-api.us-east-1.amazonaws.com/Prod/hello

Expected Output: Returns ``{"message": "Hello World from AWS SAM!"}``.

Task D: Configuring CloudWatch Error Alarms

- Look at the CloudWatch Alarm properties in `template.yaml`. It monitors the `Errors` metric of the Lambda function and triggers if errors are ≥ 1 within 60 seconds.
- In the AWS Console, navigate to CloudWatch -> Alarms.
- Verification: Locate the alarm named `sam-hello-world-dev-HelloWorldFunctionErrorAlarm` and verify that its state is `OK`.

Task E: Automating Deployment via GitHub Actions

To set up the automated CI/CD pipeline:

- Initialize Git in the root directory:

```
git init
```

```
git add .
```

```
git commit -m "Initialize SAM and CI/CD pipelines"
```

- Create a repository on GitHub.
- Configure your local repository to track your GitHub repository:

```
git remote add origin <GITHUB_REPO_URL>
```

- Configure your AWS credentials as Secrets in your GitHub repository setting:

1. `AWS\_ACCESS\_KEY\_ID` = Your access key.
2. `AWS\_SECRET\_ACCESS\_KEY` = Your secret access key.

- Push the code to the main branch:

```
git push -u origin main
```

- **Verification:** Go to the Actions tab on GitHub. Verify that the pipeline runs, executes the tests, packages the SAM template, and deploys the stack to the `dev` stage in AWS.
- **How to monitor and debug your SAM deployment:**
 - Open your repository on GitHub and select the **Actions** tab.
 - Click on the running execution (e.g., Initialize SAM and CI/CD pipelines).
 - Click on the **deploy-testing** job in the workflow diagram.
 - Expand the **Deploy to testing account** step. Here, you can monitor the live AWS SAM CLI output, showing CloudFormation resource updates in real-time.
 - If a deployment fails due to permissions, look for the execution role details in the CloudFormation console (under events) or check the output log.

Task F: Teardown

When you are finished testing, destroy the resources to avoid ongoing AWS charges.

- Run the delete command:

```
sam delete stack-name sam-hello-world-dev no-prompts
```

Note: Ensure you also delete any feature stacks created by the GitHub Actions pipeline.

Module 11: Week 5 Learning Conclusion

Throughout this Infrastructure as Code and Deployment systems automation week, we have transitioned from manual environment setups to automated delivery pipelines. Here is a summary of the core competencies and learning outcomes achieved:

- Declarative Infrastructure Foundations: Mastered AWS CloudFormation stack lifecycles, resource mappings, parameter references, and automated rollbacks.
- AWS Serverless Application Model (SAM): Configured YAML templates utilizing the `AWS::Serverless` transform headers to define functions, triggers, and resources.
- Continuous Integration & Delivery (CI/CD): Built multi-stage delivery pipelines utilizing GitHub Actions runner containers and environment secrets to automate tests, package code, and deploy serverless architectures.
- Environment Architecture Separation: Managed distinct stages (Dev, Staging, Prod) using environment parameter maps and configuration files (samconfig.toml).
- Serverless Active Auditing: Configured CloudWatch Alarms to monitor runtime exceptions, logging metrics, and error rate thresholds.
- Distributed Request Tracing: Learned to map execution paths across decoupled serverless systems and databases utilizing AWS X-Ray to isolate latency hot spots.
- Structured Operations Logging: Followed structured logging standards to improve search capabilities in CloudWatch Logs Insights.

By combining these skills, you have successfully built a production-grade automated deployment pipeline, completing the foundations of modern cloud systems engineering!

Appendix A: Glossary of Week 5 Terminology

Acronym / Term	Full Name	Definition / Cloud Role
CloudFormation	AWS CloudFormation	The base infrastructure-as-code orchestration service that reads YAML templates to manage AWS resources.

Acronym / Term	Full Name	Definition / Cloud Role
SAM	Serverless Application Model	An AWS-developed open-source framework used to simplify serverless resource deployment on CloudFormation.
Stack	CloudFormation Stack	A logical grouping of managed cloud resources instantiated from a single CloudFormation template.
Rollback	Deployment Rollback	An safety execution sequence that deletes or reverts newly updated resources if a deploy crashes.
CodePipeline	AWS CodePipeline	A continuous delivery service orchestrating testing stages, build scripts, and multi-environment promotions.
CodeBuild	AWS CodeBuild	A managed execution environment running test scripts, packaging artifacts, and compiling programs in Docker.
X-Ray	AWS X-Ray	A distributed tracing service helping developers map end-to-end user request lifecycles and latency.
Metric Alarm	CloudWatch Alarm	An alert threshold listener triggered when a designated service metric remains in breaching states.

Appendix B: Week 5 Troubleshooting & Debugging

Error Message / Issue	Likely Cause	Step-by-Step Resolution
UPDATE_ROLLBACK_FAILED stack status	A resource created by CloudFormation was manually deleted in the AWS Web Console, breaking stack synchronization.	Go to CloudWatch/CloudFormation web portal. Trigger ContinueUpdateRollback and select the manually deleted resource to skip it during the rollback, then fix the template.
Credentials not found on GitHub Runner	The repository secrets mapped in .github/workflows/pipeline.yaml are missing or misnamed in GitHub settings.	Open your GitHub repository settings $\rightarrow$ Secrets and Variables $\rightarrow$ Actions. Ensure AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY are saved correctly.
sam build docker connection fail	The local SAM CLI execution environment cannot connect to a running Docker daemon during local container testing.	Ensure Docker Desktop is active on your workstation before executing sam local invoke or sam local start-api.
InvalidParameterValueException on Deploy	The SAM template pipeline execution role does not have IAM authorization to create	Verify the IAM Policy attached to the CloudFormation execution role (TESTING_CLOUDFORMATION_EXECUTION_ROLE) grants

Error Message / Issue	Likely Cause	Step-by-Step Resolution
	specific resources.	permission for all resources.
Alarm triggering on missing data	The CloudWatch error metric alarm treats missing (null) metrics as breaches, causing false alerts when no errors occur.	Ensure your CloudWatch alarm resource properties in template.yaml include the line TreatMissingData: notBreaching.